SUMMARY OF WORK

DocMind RAG

Niusha Yaghini | 16 Aban 1405 | github.com/niusha-yaghini/docmind-rag

1. What I built

DocMind RAG is a document question-answering system. Users upload `.docx` files (or paste text); the system splits them into chunks, embeds them and indexes them in PostgreSQL. A question is answered by an OpenRouter LLM using only the retrieved passages, with citations, and every exchange is saved as history.

Stack: Django 5.2 and Django REST Framework, LangChain, PostgreSQL 16 with pgvector, Docker Compose. Django Admin and a small web interface are the user interfaces; Swagger documents the API.

2. Required scope (Level 1): delivered

Requirement	What was delivered
Django and Docker	Django project with `documents` and `qa` apps; `docker compose up --build` starts the database and the app, migrates and creates the admin user
LangChain for the flow and model calls	One LCEL chain, a custom `BaseRetriever`, the text splitter, and chat and embedding clients pointed at OpenRouter (free models, with fallbacks)
Django Admin as the UI	Add, edit and delete documents, a re-index action, visible indexing errors, read-only chunks and query logs
Complete API	Documents CRUD, re-index, search, ask and history, documented with OpenAPI and Swagger
`.docx` and full-text storage	`python-docx` reads paragraphs and tables; the text is stored in `Document.full_text`
Advanced retrieval	Hybrid search: pgvector cosine search plus PostgreSQL full-text search, merged with Reciprocal Rank Fusion
Question and answer history	`QueryLog` stores question, answer, sources, model and latency
Deliverables	Source, Docker files, README, screenshots, API docs, sample data and a technical report

3. Extras beyond the brief

1. **Web interface** (Ask, Documents, History): Django templates and vanilla JavaScript calling the same REST API, so it adds no endpoints and no build step.
2. **Citations and cited-only sources:** answers cite passages as `[n]`, and the saved sources are only the passages actually cited. A refusal lists no sources.
3. **Partial answers:** when the documents cover only part of a question, the model says what it does not know and then gives what the documents support, instead of refusing.
4. **Streaming answers:** `POST /api/ask/stream/` sends the answer as Server-Sent Events, so text appears while the model is still generating. The UI uses it.
5. **Local cross-encoder reranking:** the 20 best fused candidates are re-scored by a small local model (no API call) and the best 5 go to the LLM. The score is shown as a relevance badge in the UI and returned as `rerank_score`. If the model is unavailable, the system falls back to the plain hybrid order.
6. **Retrieval evaluation tool:** `manage.py evaluate` reports hit rate and MRR for vector only, keyword only, hybrid and hybrid with reranking, on a purpose-built corpus of 10 look-alike policy documents and 26 questions.
7. **Larger test suite:** from 31 to 59 tests, all against a real PostgreSQL with pgvector.

8. **Safety and robustness:** a prompt-injection guard, model fallbacks, failed documents kept with their error instead of lost, no LLM call on an empty index, and the reranker model baked into the Docker image.
9. **Professional workflow:** Level 1 frozen on the `level1-final` branch and the `v1.0-level1` tag; extras built on feature branches and merged through a pull request.
10. **Documentation:** a complete README (build notes, screenshots, file-by-file layout), ten screenshots, a full technical report and this summary.

4. Verification

- **Automated:** 59 tests pass, using deterministic fake embeddings and a stub LLM, so they need no network or API key.
- **Manual, with the real models:** correct factual answers with citations (500 USD home-office stipend, 14-character passwords), a correct partial answer, a correct refusal for an out-of-scope question, streaming in the web UI, and reranker scores checked inside the running container (a relevant passage scored 7.2, an unrelated one -11.3).
- **Not verified:** the evaluation tool has not yet run against the real embedding model, because the free daily quota ran out. There are therefore no hit-rate or MRR numbers, and I do not claim that reranking improved them.

5. Main challenges and solutions

Problem	Solution
LangChain's parallel step used extra Django database connections that were never closed	Made the chain sequential
Browser got HTTP 406 on the streaming endpoint (DRF refused `Accept: text/event-stream`)	A negotiation class for that endpoint, plus tests that send the same header
A question only partly covered by the documents was refused	A third prompt outcome (partial answer) with tests
Docker build could not resolve the default PyPI file host on my network	Installed the same pinned packages from a PyPI mirror, documented in the README
Free embedding quota ran out during evaluation	Embed all chunks and questions in one request, with a clear error if the quota is still exhausted

6. Limitations and next steps

- Only `.docx`; no authentication; indexing is synchronous. Free-tier models have rate and daily limits.
- Keyword search and the reranker are English only.
- Next: run the evaluation on the real model and tune from the numbers, token authentication, multilingual models, PDF support and background indexing.